

# JHQ-GPU: 写作卡

每节一张——主张、图、可引用的数字、不能说的话

分支 `fix/recall-eval-v15` — 生成于 2026-09-10 — 16 张图

这份文件是写作入口。每节一张卡, 四栏: 主张 (中英对照, 英文是待改写的草稿而非定稿措辞)、图 (用哪张、图注要点明什么)、数字 (可直接写进正文的值, 每个带出处)、红线 (不能说的话及原因——多数是本项目已经说错过一次的)。

`handbook.pdf` 和 `handbook_zh.pdf` 是背后的参考册: 前者 Part II 逐字复制每张图脚本的 `docstring`, 后者给中文提炼说明。需要查某张图的完整来龙去脉时翻它们, 写正文用这份卡。

数字以 `paper_adc2026/README.md`(证据表) 为准。[TBD] 表示还没测出来, 不要编。

## 目录

|                         |    |
|-------------------------|----|
| 写作卡 / Writing cards     | 2  |
| 1 引言 / Introduction     | 2  |
| 3 GPU-Native JHQ        | 2  |
| 4 自适应层级精化               | 5  |
| 6 实验                    | 7  |
| 7 结论 / Conclusion       | 12 |
| 还在跑的实验 (卡片里标 [TBD] 的那些) | 13 |

写作卡 / Writing cards

这份文件替代 `handbook.pdf` 作为写作入口。Handbook 的 Part I 是给 AI 的指令集 (通篇「不要说 X」),Part II 是工程自白笔记——两者都不是给人写论文用的。这里每节一张卡, 四栏:

| 栏  | 是什么                                                        |
|----|------------------------------------------------------------|
| 主张 | 这一节要论证的 1-2 句话, 中英对照。英文是待改写的草稿, 不是定稿措辞。                    |
| 图  | 用哪张图, 图注要点明什么                                              |
| 数字 | 可以直接写进正文的值, 每个都带出处                                         |
| 红线 | 三种标记: 不可 = 不能这么写; 应当 = 该这么写; 注意 = 一条必须尊重的事实。多数是本项目已经说错过一次的 |

数字以 `README.md`(证据表) 为准。[TBD] 表示还没测出来, 不要编。

1 引言 / Introduction

主张

JHQ 的两级量化把打分拆成便宜的主近似和准确的残差精化, 这个结构在 GPU 上留下两个未解问题: 结构该怎么执行, 以及一个工作负载实际需要多少精化。我们回答这两个, 并把答案的边界测出来。

JHQ’s two-level quantisation splits scoring into a cheap primary approximation and an accurate residual refinement. On a GPU this leaves two open questions —how the structure should execute, and how much refinement a workload actually needs. We answer both, and measure where the answers stop holding.

图无 (引言不放图)。

数字 (引言里可以点的, 全部有出处)

- 六个数据集, 最大 1780 万向量  $\times$  1024 维 (`README.md` 数据集表)
- 三个 GPU 基线:IVF-RaBitQ、CAGRA(fp32/int8)、IVF-PQ
- 主表分解: 每子空间 256 项  $\rightarrow$  32 项, 精确 (代数推论, `fig_lut`)
- 标定规则:32 个改变了预算的配置上 **1.044–2.653** $\times$ , 中位回本 **1.75** 批 (`fig_economics`)

红线

- 不可 不要写「JHQ-GPU 是最快的 GPU ANN 方法」。写工作区域:JHQ 在四个可比数据集上到 R=0.95 领先, 高召回尾部让位。
- 不可 不要在引言承诺 CPU 加速比——那个数还是 [TBD]。
- 不可 不要用「25 $\times$ 」形容  $\alpha$  的跨度。写「4 到至少 200(`nprobe`=128)」, 上端是下界。

3 GPU-Native JHQ

3.1 设计总览

主张

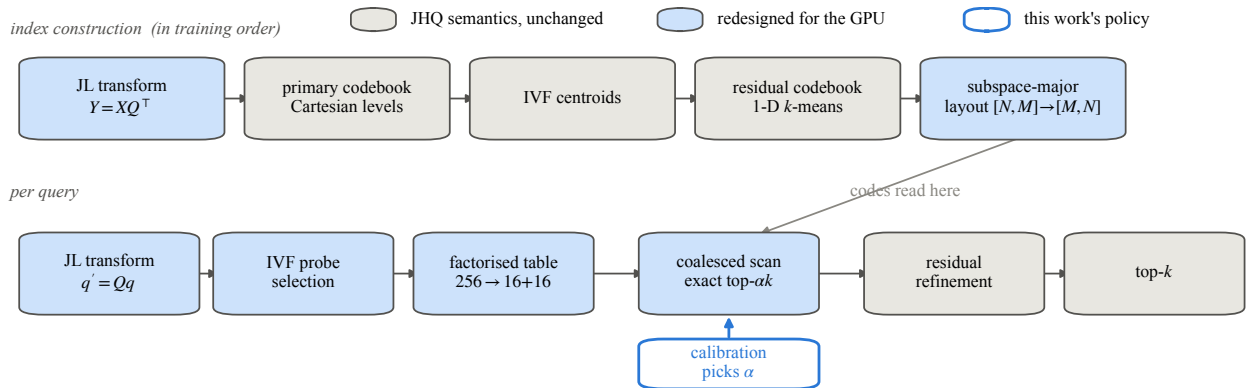


图 1: **fig\_pipeline**. the JHQ-GPU pipeline, shaded by what changed 完整说明与全部注意事项见第二部分 `fig_pipeline.py`。

我们保留 JHQ 的层级量化语义不变, 只重新设计它的物理表示和主距离路径。表示决定哪些距离计算可以被精确重组; 精化预算决定近似过滤发生多少。

We preserve JHQ's hierarchical quantisation semantics and redesign only its physical representation and primary-distance path. The representation determines which distance computations can be reorganised **exactly**; the refinement budget determines how much approximate filtering happens.

图 `fig_pipeline`。图注要点明: 着色区分「GPU 重设计」和「原样保留的 JHQ 语义」, 并且构建行是代码真实的训练顺序。

数字记号:  $x \rightarrow y = Qx, q' = Qq, Q \in \mathbb{R}^{(d \times d)}, M$  个子空间, 主码  $c_m$ 。

红线

- 注意 **JL**/正交变换是方阵, 不是降维。
- 注意 残差是  $r = y - \hat{y}_{\text{primary}}$ , 不是 **IVF** 质心残差。这是第 3 节最容易被误读的一点。
- 不可 不要把训练顺序画成「残差码本在 **IVF** 质心之前」。真实顺序是 **JL**  $\rightarrow$  主码本  $\rightarrow$  主码编码  $\rightarrow$  **IVF** 质心  $\rightarrow$  残差码本 (`JHQ_TRAIN_PHASE`)。残差码本用采样, 不等全部分配完毕——画反会暗示一个不存在的依赖。

### 3.3.1 笛卡尔分解的主距离表 [核心] 最强贡献

主张

因为 JHQ 的主码本本身就是一维 Lloyd-Max 层级的笛卡尔积, 平方欧氏距离在互不相交的坐标上可加, 所以每子空间的查询表可以精确分解为两张 16 项表:  $T_m[c] = T_m^{\text{hi}}[c \gg 4] + T_m^{\text{lo}}[c \bmod 16]$ 。256 项变 32 项, 距离逐位不变。

Because JHQ's primary codebook is itself a Cartesian product of one-dimensional Lloyd-Max levels, and squared Euclidean distance is additive over disjoint coordinates, each subspace's query table factorises **exactly** into two 16-entry tables:  $T_m[c] = T_m^{\text{hi}}[c \gg 4] + T_m^{\text{lo}}[c \bmod 16]$ . 256 entries become 32, and no distance changes.

图 `fig_lut(机制)`+ `fig_lutgroups(为什么是这个粒度、值多少)`。

数字

- 每子空间  $256 \rightarrow 16+16 = 32$  项; 表构建工作 **16 $\times$**  更少 (代数, `fig_lut`)

- 代价: 每子空间一次查表变两次
- 实测收益强烈依赖  $M$ :  $M=96$  时  $-4\%\sim+12\%$ ,  $M=384$  时  $+23\%\sim+53\%$ (data/lut\_groups.log)
- 机制: 256 项表是  $M \times 256 \times 4$  字节—— $M=96$  是 98 KiB(共享内存放得下),  $M=384$  是 **393 KiB**(放不下)
- 为什么是对半分:  $G=2$  在全部 8 个配置上胜过  $G=1/4/8$ ;  $G=4$  和  $G=8$  的表更小却按加载数  $(2/4/8)$  成比例更慢

### 红线

- 不可 不要拿代数上的  $8 \times$  条目缩减冒充加速比。这是本项目踩过的坑。
- 不可 不要说「 $+6\%$  到  $+14.5\%$ , 无一为负」——那 28 个格子全是  $M=96/128$ ,  $M=384$  上低估四倍, 而  $M=96$  有一格是负的。
- 不可 不要断言「没有别的量化器能做分解」。写: 精确分解来自这一笛卡尔表示, 对比的基线在其当前表示下不具备。
- 注意 必须引 **FastScan** 和 **Quicker ADC**(相关工作)。它们的目标同样是把 ADC 表压小到能进 SIMD 寄存器。区别是: 它们靠改量化器换小表 (**4 位码、更粗码本、拿精度换**), 我们什么都没改——码还是 8 位, 距离逐位相同。不引会被懂行的审稿人当成已知工作。
- 注意 代数精确  $\neq$  浮点逐位相同, 也  $\neq$  并列顺序确定。

### 3.3.2 合并访存的主扫描与打包访问

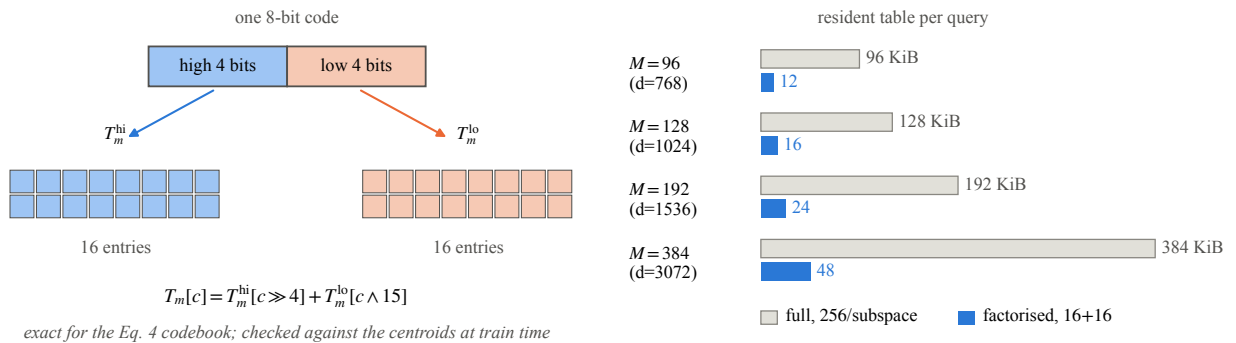


图 2: **fig\_lut**. the Cartesian factorisation, and what it saves per query 完整说明与全部注意事项见第二部分 **fig\_lut.py**。

### 主张

子空间主序布局让一个 warp 读到连续字节; 而可容许性规则  $D_s \mid B$  在  $B=8$  时逼出每维一比特, 于是四个子空间共享一次 32 位加载。前者是合并访存, 后者是指令数——同一个结构性质付了两次红利, 而且两笔收益相乘。

The subspace-major layout lets a warp read contiguous bytes; and the admissibility rule  $D_s \mid B$  at  $B=8$  forces one bit a dimension, so four subspaces share a 32-bit load. The first is coalescing, the second is **instruction count** —one structural property paying twice, and the two gains multiply.

图 **fig\_layout**(两个效应分开)+ **fig\_lutgroups(b)**(相乘的证据)。

### 数字

- 字打包 **+30% 到 +48%**(results/front6/NEGATIVES.md)

- 两笔收益相乘不重叠: 分解在 byte 布局值 1.13/1.24/1.36, 在 packed 上 1.10/1.23/1.53(data/lut\_groups.log)

## 红线

- 不可 绝不要写「128 字节 cache line 浪费 75%」。Pascal 之后全局访问按 32 字节 sector 服务, 转置之后取的本来就是要用的 sector。这个错误说法曾写进本项目多处文档。
- 不可 字节布局已经合并的情况下, 不要暗示打包减少了传输字节。它减少的是加载指令、寻址和循环迭代。
- 不可 不要把转置说成新颖性——转置布局 FAISS-GPU 和 cuVS 早就在用。写「必要的标准 GPU 工程」。
- 应当 按查询数发射、游标删除属于实现修正, 单独标注, 不与贡献并列。

## 3.4 收尾: 剩下的策略问题

### 主张

执行路径接受一个幸存者预算  $ck = \alpha \cdot k$ , 但不决定这个预算该多大。

The execution path accepts a survivor budget  $ck = \alpha \cdot k$  but does not determine how large it should be.

红线 不可 不要用泛泛的总结收尾。第 3 节必须以这个未解参数结束, 才接得上第 4 节。

—

## 4 自适应层级精化

### 4.1 动机

#### 主张

原版 JHQ 把  $\alpha$  留给外部指定, 而足够的预算跨工作负载差一个数量级以上: openai3-3072 从  $\alpha=4$  起就饱和, arxiv-768 到  $\alpha=200$  仍在改善。单一常数因此要么浪费要么有损, 而是哪一种在跑完扫描之前无从得知。

Original JHQ leaves  $\alpha$  externally specified, and the sufficient budget spans more than an order of magnitude across workloads: openai3-3072 is saturated from  $\alpha=4$  while arxiv-768 is still improving at  $\alpha=200$ . A single constant is therefore either wasteful or lossy, and which one cannot be known without running the sweep.

图 fig\_alpha(a)。

#### 数字

- 足够预算 4 到至少 200(nprobe=128); 上端是下界, arxiv 在网格最大处仍在降 (data/alpha6.log)
- 纵轴是 ivf\_recall - recall, 路由损失已剔除

## 红线

- 不可 不要写「25×」。上端删失, 一个精确比值既不准确又过度精确。
- 不可 不要用 1-recall 当排序损失——那里面大半是路由损失, 而  $\alpha$  够不到没被打开的桶。
- 注意 alpha6.log 是 DIAG 构建, 它的 QPS 列不能用 (诊断回读改变计时); 召回类的量不受影响。

#### 4.2-4.3 输出稳定性判据与选择过程

##### 主张

规则问的是系统自己能回答的问题: 降低预算会不会改变我返回的东西。取  $S$  条本批查询, 在充裕的  $\alpha_{\max}$  下答一次作为参考, 再在更小的  $\alpha$  上答, 取采样 top-k 仍在容忍内一致的最小者。不用 ground truth, 不用学习到的预测器。

The rule asks the only question the system can answer without labels: would a smaller budget change what I return. Take  $S$  of the batch's own queries, answer once at a generous  $\alpha_{\max}$  as the reference, answer again at smaller  $\alpha$ , and keep the smallest whose sampled top-k still agrees within tolerance. No ground truth, no learned predictor.

图 fig\_rule。图注要点明: 右图是从日志解析的一次真实标定, 不是示意。

##### 数字

- 判据:  $D(\alpha) = \sum_q [k - |R_{\alpha}(q) \cap R_{\max}(q)|] \leq \epsilon, \epsilon$  是整数槽位数 (不是分数)
- 默认  $S=32, \epsilon=1$ ; 网格  $\{100, 64, 32, 16, 8, 4, 2\}$ , 走二分, 7 点网格 3 次探测
- 容忍度扫描 (vogue, nprobe=512): 0 槽  $\rightarrow \alpha=100$  无增益; 1 槽  $\rightarrow \alpha=64, 1.12\times$ ; 2 槽  $\rightarrow \alpha=32, 1.27\times$  但让出 **0.0035**(data/alpha\_fast.log)

##### 红线

- 不可 不要说「停在第一次拒绝, 所以单边」。那描述的是可选的线性模式。默认是二分, vogue 上第一次探测就被拒、搜索继续往上 (16 拒绝  $\rightarrow$  64 接受  $\rightarrow$  32 拒绝  $\rightarrow$  选 64)。
- 应当 该写的是: 二分返回采样判据能接受的最小网格  $\alpha$ , 上界是  $\alpha_{\max}$ ; 它成立依赖「接受谓词随  $\alpha$  单调」, 而这一条可以论证——精确 top-ck 选择给出嵌套候选集, 所以不一致槽位数随  $\alpha$  单调不增。
- 注意 必须做相关工作定位: 基于采样的参数调优不新 (FAISS ParameterSpace; 自适应提前终止, Li et al. SIGMOD 2020)。我们的区别是无标签、无预测器的输出稳定性, 直接作用于 JHQ 外部指定的  $\alpha$ , 并对着它取代的那个穷举扫描做了验证。
- 不可 不要说规则「绝不会选得太小」。

#### 4.4 标定成本与回本

##### 主张

规则用一次前置成本换稳态吞吐, 所以每个增益都以后续批次数为条件。在 32 个真正改变了预算的配置上, 增益全部大于 1 (1.044–2.653 $\times$ ), 回本 0.5 到 21.0 批、中位数 1.75。成本最大的地方恰好是增益最小的地方, 因为两端都随 nprobe 同向变化。

The rule trades a one-off cost for steady-state throughput, so every gain is conditional on the number of subsequent batches. Over the 32 configurations where the budget actually changed, the gain is above one in all of them (1.044 $\times$  to 2.653 $\times$ ) and payback runs 0.5 to 21.0 batches with a median of 1.75. The cost is largest exactly where the gain is smallest, because both ends move with nprobe.

图 fig\_economics。

##### 数字

- $B^* = T_{\text{cal}} / (T_{\text{fixed}} - T_{\text{rule}})$ ; 标定 **2.0–47.5 ms**
- 32 个改变了预算的配置: 增益 **1.044–2.653**×, 回本 **0.5–21.0** 批, 中位数 **1.75**
- 召回代价 **0.0000** 到 **0.0048**, 最大是 bge-m3 在 nprobe=1024

## 红线

- 不可 不要报「**0.6–178.6** 批」或「中位数 **2.7**」。arxiv 在 nprobe $\geq$ 128 时规则返回  $\alpha_{\text{max}}=100$ , 而固定臂跑的也是 100——两条臂是同一个配置, gain 是重复计时的  $\pm 1.2\%$  波动, 178.6 是 1/噪声。那四行要单独归类为「预算未改变」。
- 不可 不要说 bge-m3 是「唯一真实代价」。地板放到  $1e-4$  后 openai3-1536 的 -0.0028 同样是代价。
- 注意 增益与回本的反相关大半是定义性的 ( $B^* = (T_{\text{cal}}/T_{\text{rule}})/(g-1)$ ,  $g \rightarrow 1$  时发散)。图的价值在量级, 不是机制发现。
- 注意 复用假定查询分布稳定, 而漂移没测过。要么写成局限, 要么定义重标定周期 H 并摊  $T_{\text{cal}}/H$ 。

## 6 实验

### 6.1 设置与协议

#### 红线 (这一节全是红线)

- 应当 统一计时边界: 主机查询进  $\rightarrow$  主机结果出
- 应当 **Recall@10**, 全程 **k=10**;  $\alpha$  与 k 由构造耦合, 跨 k 推广性 [TBD]
- 应当 前沿曲线报稳态吞吐, 标定成本单独报, 通过回本分析纳入
- 应当 等召回比值是相邻实测点之间的对数线性插值, 要标明是插值, 绝不外推
- 应当 粗量化器  $JHQ\_N\_TRAIN = 39 \times nlist$ , 每个数据集都是
- 应当 可重复性有两个地板, 不能互换: 跨 build 是 **1e-3**(训练不可复现, 同一二进制三次冷启动 0.9852/0.9842/0.9849); 同一索引上的配对比较是 **1e-4**(只有 top-ck 并列打破在变)
- 注意 一个扫描端点不是架构召回上限
- 不可 不要把修复前和修复后的 JHQ 点混成一条前沿

### 6.2 端到端 Recall-QPS —— RQ1

#### 主张

各方法占据不同的召回-吞吐区域, 而 JHQ 的优势随维度增长: 在四个可比数据集上它到  $R=0.95$  都领先,  $d=1536$  时全程  $1.36\text{--}2.35\times$ , 而在高召回尾部让位。

The methods occupy different recall-throughput regions and JHQ's advantage grows with dimensionality: it leads on all four comparable datasets through  $R=0.95$ , by  $1.36\times$  to  $2.35\times$  across the whole range at  $d=1536$ , and gives way in the high-recall tail.

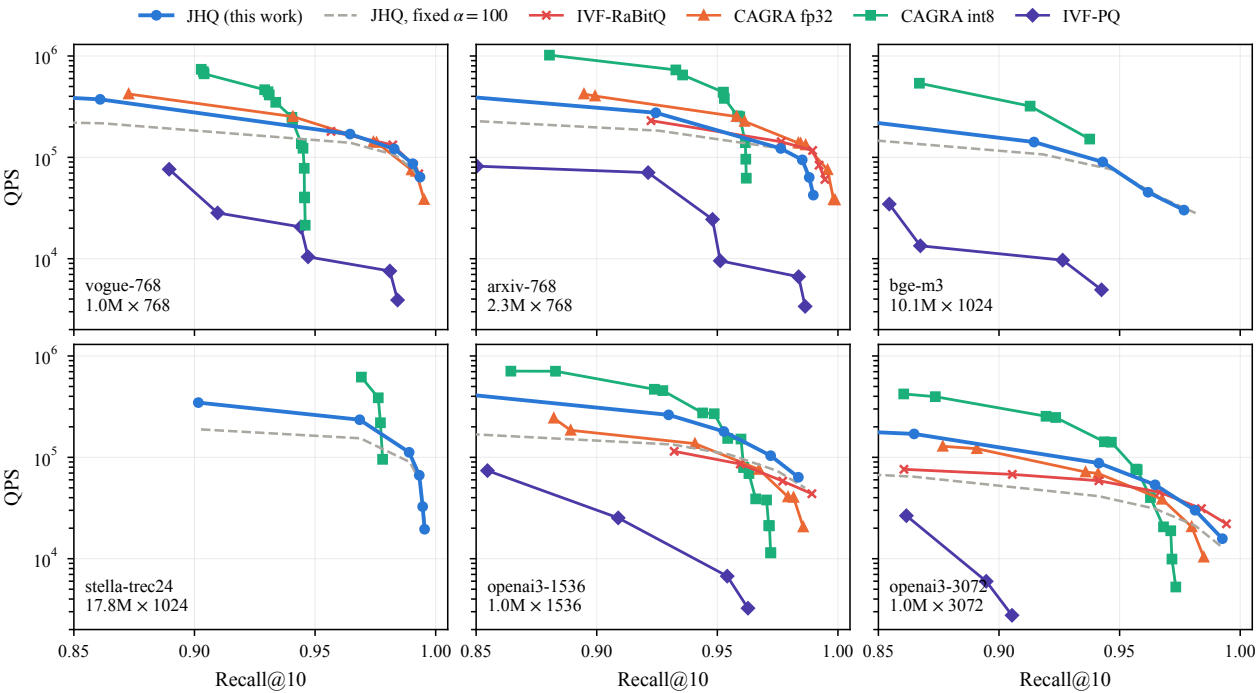


图 3: **fig\_frontier**. six-panel recall-QPS frontier 完整说明与全部注意事项见第二部分 *fig\_frontier.py*。

图 *fig\_frontier*。

数字 (JHQ ÷ IVF-RaBitQ, 等召回插值)

| 数据集                 | d    | R=0.90       | R=0.95       | R=0.98       |
|---------------------|------|--------------|--------------|--------------|
| vogue-768           | 768  | 1.24×        | 1.02×        | 0.94×        |
| arxiv-768           | 768  | 1.29×        | 1.02×        | 0.81×        |
| <b>openai3-1536</b> | 1536 | <b>2.35×</b> | <b>1.98×</b> | <b>1.36×</b> |
| openai3-3072        | 3072 | 1.82×        | 1.36×        | 0.93×        |

- CAGRA fp32 和 IVF-RaBitQ 在 **bge-m3** 和 **stella** 上建不起来 (1010 万和 1780 万 × 1024 维, 分配失败)

红线

- 注意 没有普适赢家, 也不能仅凭维度推断因果。
- 不可 不要把曲线终点画成能力上限 (阴影已删)。
- 应当 「建不起来」 必须指明所测的配置和库路径, 不是 「该算法永远无法索引」。
- 不可 CPU JHQ 要单独一张等召回表, 不要和 GPU 前沿共用坐标轴 (差两三个数量级)。当前 [TBD]。

6.3 自适应精化评估——RQ2

主张

规则在 nprobe=128 上四个配置中三个精确命中扫描找到的  $\alpha$ ; 第四个不是规则出错, 是它的饱和点在规则自己的  $\alpha_{\text{max}}$  之上。但样本量的选择不能是一个常数: 留出验证显示 S=32 在一个数据集上够、在另一个上远不够。

The rule recovers the swept  $\alpha$  in three of four configurations at nprobe=128; the fourth is not an error but a saturation



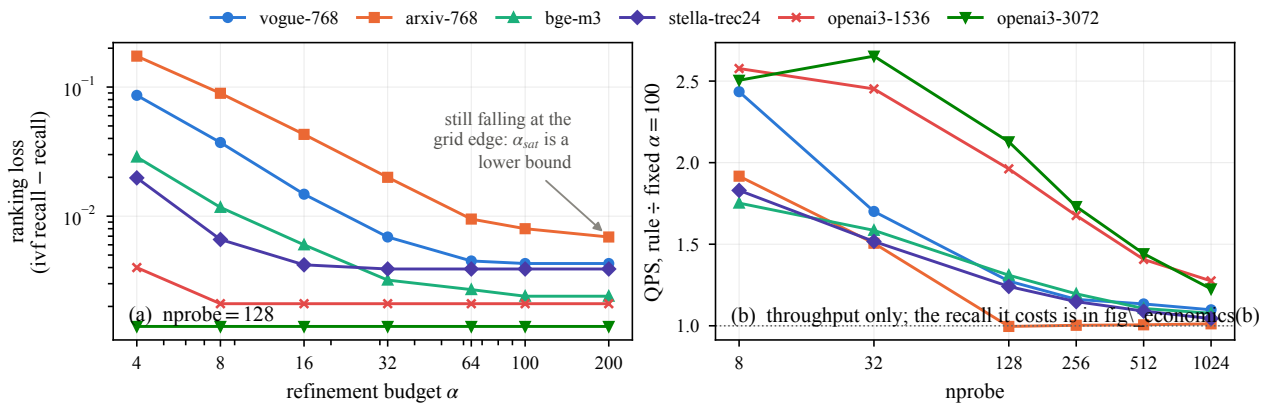


图 4: **fig\_alpha**. ranking loss vs alpha; what the rule is worth 完整说明与全部注意事项见第二部分 *fig\_alpha.py*。

point above the rule's own  $\alpha_{\text{max}}$ . The sample size, however, cannot be a constant: held-out validation shows  $S=32$  sufficing on one dataset and falling well short on another.

图 *fig\_calibration*(三格)+ *fig\_alpha*。

数字

- 2000 次重采样、只在留出查询上打分:  $S=32$  时 openai3-3072 有 **76%** 落在  $1e-3$  内, vogue-768 只有 **27%**(均值 0.0033、p95 0.0110)
- openai3-3072 在  $S=64$  达 **98%**; vogue-768 到  $S=128$  仍有 **11%** 落在外面

红线

- 不可 不要写「 $S=32$  是拐点」。那来自每个  $S$  一次抽样、且在包含标定样本的整批上打分。写「拐点是工作负载的性质, 按声明的风险给  $S$ 」。
- 不可  $nprobe=512$  没有跑过穷举扫描, 不得复用  $nprobe=128$  的答案。
- 不可 容忍度那三个点必须全部来自二分(旧图混进了线性变体的一行, 三根柱子两个算法)。

#### 6.4 表示与执行消融——RQ3

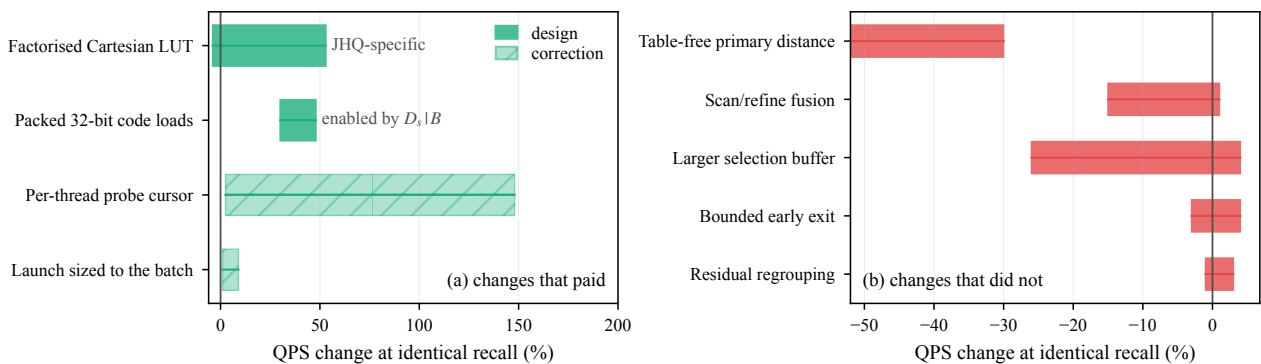


图 5: **fig\_ablation**. what paid, and what did not 完整说明与全部注意事项见第二部分 *fig\_ablation.py*。

主张

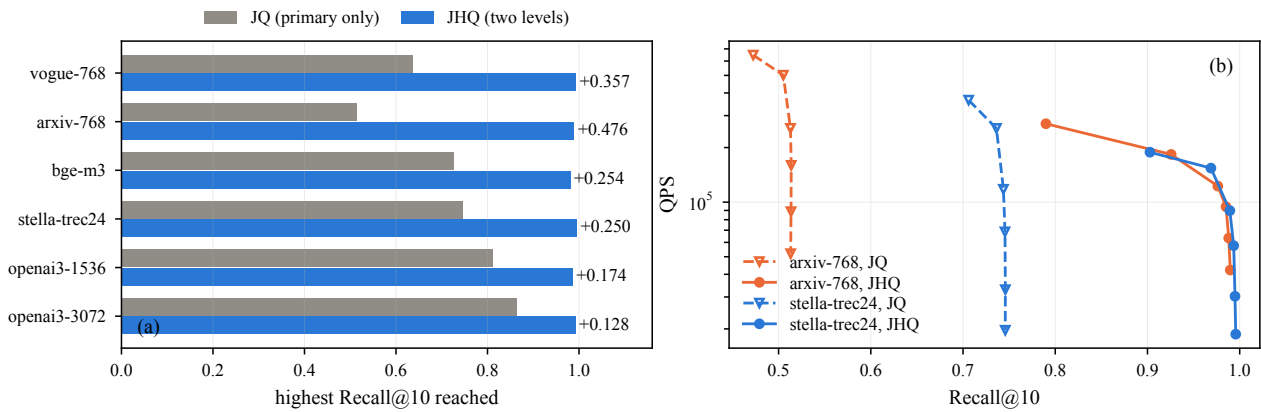


图 6: **fig\_hierarchy**, what the second level buys 完整说明与全部注意事项见第二部分 *fig\_hierarchy.py*。

四个独立实验指向同一机制: 主扫描受指令发射限制, 不受表大小限制。免表距离搬更少内存却慢 30–52%; 字打包搬同样字节却快 30–48%; 更小的表输给更大的表 (G=4 慢 4–36%, G=8 慢 10–61%); 而删掉内层循环约 30% 的冗余边界测试换来最高 +148%。

Four independent experiments point at one mechanism: the primary scan is issue-bound, not table-size-bound. The table-free distance moves less memory and is 30–52% slower; the packed load moves the same bytes and is 30–48% faster; smaller tables lose to a larger one (G=4 by 4–36%, G=8 by 10–61%); and removing about 30% of the inner loop's redundant boundary tests is worth up to +148%.

图 *fig\_hierarchy*(6.4.1)、*fig\_lutgroups*(6.4.2)、*fig\_ablation*(6.4.3)。

## 数字

- 残差层级: 实测最高召回, 完整 JHQ 减仅主码 = **+0.128(openai3-3072)** 到 **+0.476(arxiv-768)**(data/hierarchy\_ablation.log + paper\_fronts.log)
- 粒度: G=2 全胜八个配置; vogue nprobe=512 相对 G=2 的比值  $G=1/4/8 = 0.90/0.64/0.39$
- 四种粒度召回完全一致 (恒等式要求如此——这是先于计时的正确性检查)

## 红线

- 应当 把最大值称为「实测最高召回」, 不是架构上限。
- 注意 这个消融测的是层级, 不是一个调优完备的独立 JQ 系统。
- 不可 不要把不同批次实验的百分比加起来推断合成加速比——用 6.4.2 那个  $2 \times 2$ 。
- 不可 *fig\_ablation* 里自适应  $\alpha$  不该出现 (它是策略结果、对固定  $\alpha=100$  而非等召回、且带召回代价)。
- 应当 新颖性和效应量是两个轴: 打包的实测加速比分解大, 如实报告, 不要改新颖性标签去迁就性能。
- 注意 游标 (**+2.5% 到 +148%**) 是整个消融里最大的单个数字, 比分解 (最高 +53%) 和打包 (+48%) 都大。它是实现修正, 所以不进贡献层级; 但把它的数字藏掉是另一个方向的错误——骨架自己写了「如实报告两者」, 现在的处理只做了「不改标签」这半句。
- 应当 正确位置是第三条路: 6.4.3 保留它的斜纹柱和真实区间 (不称新颖), 同时在 §6.5 把它当机制证据讲。它单独改的就是指令数, 别的都没动, 所以是四条证据里最直接的那条。

## 6.5 机制分析与负面结果——RQ4

### 主张

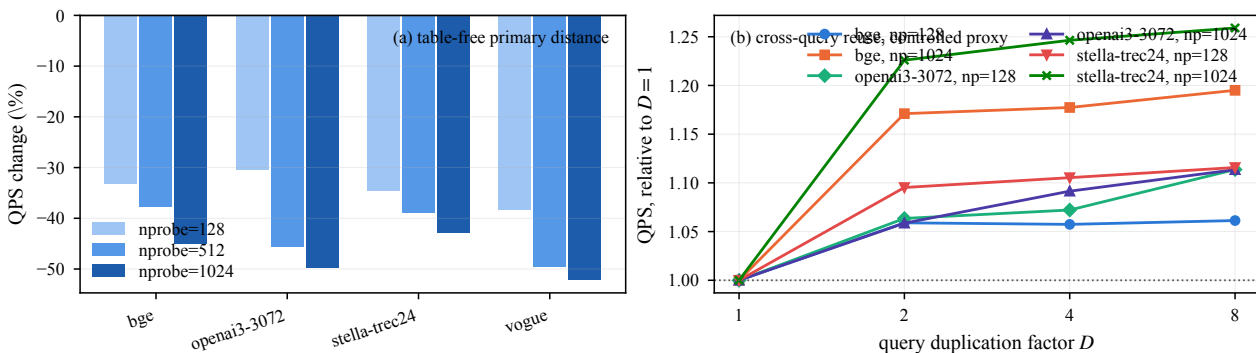


图 7: **fig\_negatives**, the table-free distance, and reuse as a controlled proxy 完整说明与全部注意事项见第二部分 *fig\_negatives.py*。

更少的表字节不等于更快的执行。四个实验分别改变访存量 and 指令数, 而结果一致指向后者: 这个扫描受指令发射限制。按字节数做的直觉推理在这里给出错误答案。

Fewer table bytes does not mean faster execution. Four experiments vary memory traffic and instruction count separately, and all four point at the second: this scan is issue-bound. Reasoning from byte counts gives the wrong answer here.

图 *fig\_negatives*(免表与复用代理)+ *fig\_lutgroups(a)*(更小的表更慢)。

数字——四条证据, 一个机制

| 实验                  | 搬的内存 | 指令     | 结果            | 出处                              |
|---------------------|------|--------|---------------|---------------------------------|
| 免表主距离 (符号内积)        | 更少   | 更多     | -30% ~ -52%   | data/v54.log                    |
| 字打包                 | 相同   | 更少     | +30% ~ +48%   | results/front6/<br>NEGATIVES.md |
| 更细的表 G=4            | 更少   | 更多     | -4% ~ -36%    | data/lut_groups.log             |
| 更细的表 G=8            | 更少   | 更多     | -10% ~ -61%   | data/lut_groups.log             |
| 每线程 <b>probe</b> 游标 | 相同   | 少约 30% | +2.5% ~ +148% | results/front6/<br>NEGATIVES.md |

- 跨查询复用增益在 **D=2** 就饱和 (data/qdup.log)
- 阶段计时佐证: *build\_byte\_lut* 只占一个批次的 **0.2%~2.3%**, 所以分解的收益不可能来自「表构建省 16×」; 它来自扫描对一张更小、无 bank 冲突的表的访问 (data/k\_and\_stages.log)

游标那一条值得单独写清楚, 因为它是四条里唯一只改指令数的。 *scan\_ivf\_exact\_kernel* 里每个线程需要知道自己的候选属于哪个 probe。 *p* 是个 per-thread 寄存器, 本该跨 chunk 携带, 但只有持有该 **chunk** 最后一个候选的线程写过它——其余 1023 个线程的 *p* 一直是 0, 每个 chunk 从零重走一遍 probe 边界。代价是每候选约 *nprobe*/2 次边界测试, 而不是整个扫描共 *nprobe* 次: **stella** 在 **nprobe=128** 上是每线程 **14,308** 次对 **128** 次, 约占内层循环指令的 **30%**。修法精确的理由: 每个线程拥有 *lt = chunk + tid*, 每 chunk 增长 BLOCK, 所以它的 probe 索引单调, 私有游标给出同样的候选、同样的距离、同样的顺序。

红线

- 注意 重复查询实验是受控复用代理, 不是上界。它固定调度只变查询共性; 真正的重写还会改调度。
- 应当 L2 是推断的就写「与 L2 已吃掉大部分复用相符」, 不要直接归因。
- 注意 单卡观察不构成跨架构定律。

## 6.6 批大小敏感性——RQ5 / 6.7 构建与显存——RQ6

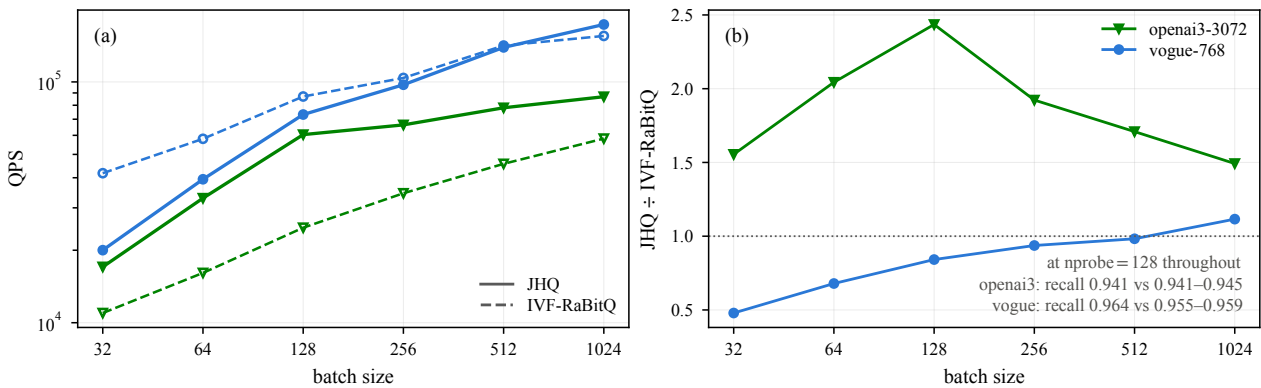


图 8: **fig\_batch**. throughput and the JHQ/RaBitQ ratio against batch 完整说明与全部注意事项见第二部分 *fig\_batch.py*。

### 数字

- 批比值在两个数据集上朝相反方向移动 (*fig\_batch*)
- 构建 (秒): JHQ **1.1–14.3**; CAGRA int8 9–75; IVF-PQ 6.4–43; IVF-RaBitQ 4.5–14.0(能建的地方)(*fig\_build*)
- 显存: IVF-RaBitQ 在四个有实测的数据集上比 JHQ 小 **22–39%**
- 未归因余量在 vogue 占 **45%**、stella 占 **5%** —— 固定开销被规模摊薄 (*fig\_memory(b)*)

### 红线

- 注意 现有批扫描是固定 **nprobe=128**, 不是等召回。vogue 上 JHQ 是在更高召回上被计时的 (0.9645 对 0.9549–0.9592)。等召回版本在跑。
- 不可 不要靠复制约 1000 条真实查询构造「独立的 10k 工作负载」——复制本身就能靠 L2 复用抬高 4–26%。
- 应当 说「JHQ 处在一个不同的 显存-精度-吞吐工作区」, 不要说普遍显存优势。
- 应当 显存缺口那一段叫「**other / unaccounted**」, 不是「context + allocator」——后者是预期成因, 没被单独测过。
- 注意 构建时间: train 有缓存、add 没有。取  $\max(\text{train}+\text{add})$  会把 13.4 s 报成 22 s。

## 7 结论 / Conclusion

### 主张

JHQ 的笛卡尔主表示允许对主距离求值做精确重组, 而这项收益随子空间数增长; 它留给外部指定的精化预算可以从工作负载自身选出, 但样本量必须按工作负载给定风险。两者合起来在特定的召回、维度和批大小区域上给出有竞争力的吞吐, 并伴有明确的显存与构建代价。

JHQ's Cartesian primary representation admits an exact reorganisation of primary-distance evaluation, and the benefit grows with the number of subspaces. The refinement budget it leaves externally specified can be selected from the workload itself, though the sample size must be given per workload against a stated risk. Together they deliver competitive throughput in specific recall, dimensionality and batch regions, with explicit memory and build-time costs.

红线

- 不可 结论里不要出现引言没承诺过的主张。
- 不可 CPU 加速比在必需基线落地之前保持 [TBD], 不要在结论里补一句「显著快于 CPU」。
- 应当 跨 k、漂移工作负载、第二块 GPU 都未测量——明确写成局限, 不要暗示已覆盖。

—

还在跑的实验 (卡片里标 [TBD] 的那些)

| 项 | 内容                                                               | 会影响哪张卡             |
|---|------------------------------------------------------------------|--------------------|
| 2 | CPU JHQ 基线 (线程扫描中;208 线程档<br>段错误)                                | 1、6.2、7            |
| 6 | k 敏感性 ( $k \in \{1,10,100\}/\{1,10,20\}; \alpha \times k$<br>交叉) | 6.1、7              |
| 7 | 等召回批扫描 ( $\text{batch} \times \text{nprobe} \times$ 两系<br>统)     | 6.6                |
| 8 | 分阶段计时 (8 段对 $\alpha \times \text{nprobe}$ )                      | 6.4、以及 §3↔§4 的实证连接 |